

Selecting Securities (Programmers)

Due to the specialness of securities (commodities), both on the technical level and on the business-logic level, there are several things you must understand/keep in mind before working with them.

In a nutshell, it boils down to the following: There normally are technical IDs, and there are business logic IDs. And sometimes, as here, there are pseudo-technical IDs, which in fact are business-logic IDs.

Please read the document “ID Layers in GnuCash” (folder `xsec`) first before you move on. It is important to understand the (non-)difference between GnuCash’s technical and business-logic security IDs described there before reading the next section; otherwise it will probably confuse you.

Specifying IDs in the API

I will be succinct, because you are a developer and therefore should be able to read code.

ID Types

The module “Base” contains several ID types, most of them being genuine technical ones.

You will, however, also find:

- `GCshCmntyID`
 - `GCshCurrID`
 - `GCshSecID`
 - * `GCshSecID_Exchange`
 - * `GCshSecID_MIC`
 - * `GCshSecID_SecIDType`

`GCshSecID_SecIDType`, e.g., might stand for something like. “ISIN:DE000BASF111”, and `GCshSecID_Exchange`, might stand for something like. “EURONEXT:SAP”.

You will also find additional documentation about what they mean in the file `GCshCmntyNameSpace.java`.

The point I want to make is: Do as if you did not know what exactly a `GCshSecID_xyz` looks like; as if it were a UUID just as with all other GnuCash objects. You don’t “read” `14305dc80e034834b3f531696d81b493` (like: “understand” or “interpret” it), do you? You just know it’s there, get it from one method’s output and put it into another one’s args. In short: *ignore its semantics!*

Getting Security IDs

In short: The lib does not provide methods which you can get *IDs* (yet). Instead, it provides methods which you can get *objects* with, which in turn, obviously, have an ID.

The class `GnuCashFileImpl` contains some methods for this:

- Get *one* commodity object by something:
“Something” being an identifier, i.e. an ID object from above, or a list of its constituents, or another (usually unique) criterion.

- `getCommodityByID(...)`
- `getCommodityByQualifID(...)` (several variants)
- `getCommodityByNamSpcCode(...)` (several variants)
- `getCommodityByXCode(...)`
- `getCommodityByNameUniq(...)`

They all return one `GnuCashCommodity` object the ID of which you can get with the method `getQualifID()`.

- Get *several* commodity objects by something:
 - `getCommoditiesByName(...)` (two variants)
 - `getCommodities()`

They all return a *list* of `GnuCashCommodity` objects.

In addition to this, in module “Specialized Entities”, you will find the class `GnuCashFileExtImpl` that contains specialized security-variants of the above-mentioned methods:

- `getSecurityByID(...)`
- `getSecurityByNamSpcCode(...)` (several variants)
- `getSecurityByXCode(...)`
- `getSecurities()`

Have a look at module “API Examples”, program `GetCmdtyInfo` for a simple example on how to use them.

You will also find a more elaborate variant of it (with better code encapsulation) in module “Tools”, program `GetSecList`.

Last not least: Have a look at the test cases for `GnuCashCommodityImpl` and `GnuCashSecurityImpl`.

Selecting a Security Object with an ID

This section overlaps with the previos one, and there is a reason for it:

Once you have the `GnuCashCommodity`/`GnuCashSecurity` object (or its ID, resp.), things are just as easy and straight forward as with any other entity: Either you already have the object (congrats), or you just have its ID (from a mythical external source), and then you use:

```
GnuCashFileImpl.getCommodityByID(cmdtyID) or
GnuCashFileExtImpl.getSecurityByID(secID).
```

Look at the example program in module “API Examples”, it is called `GetCmdtyInfo`.

(You will also find a more elaborate version of it in module “Tools”, called `GetSecInfo` there, as it is specialized on securities.)

Notice that the entity “security” (as opposed to “commodity” which it inherits from) is only defined in the module “API Specialized Entities”.